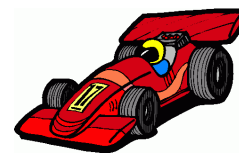


Quicksort



Considere o problema da ordenação discutido em [outro capítulo](#). O problema consiste em [permutar](#) os elementos de um vetor $v[0 \dots n-1]$ para colocá-los em [ordem crescente](#), i.e., rearranjar os elementos do vetor de tal modo que tenhamos $v[0] \leq \dots \leq v[n-1]$.

O outro capítulo analisou alguns algoritmos simples para o problema. O presente capítulo examina um algoritmo mais sofisticado — o Quicksort — [inventado por C.A.R. Hoare](#) em 1962. Em algumas raras [instâncias](#), o Quicksort é tão lento quanto os algoritmos simples, mas em geral é muito mais rápido. Mais exatamente, o algoritmo é [linearítmico](#) em média, embora seja quadrático no pior caso.

Eis a ideia básica do algoritmo: se todos os elementos pequenos estiverem na metade esquerda do vetor e todos os elementos grandes estiverem na metade direita, basta ordenar cada metade independentemente.

Usaremos duas abreviaturas ao discutir o algoritmo. A expressão “ $v[i \dots k] \leq x$ ” será usada como abreviatura de “ $v[j] \leq x$ para todo j no conjunto de índices $i \dots k$ ”. A expressão “ $v[i \dots k] \leq v[p \dots r]$ ” será interpretada como “ $v[j] \leq v[q]$ para todo j no conjunto $i \dots k$ e todo q no conjunto [p...r](#)”.

Sumário:

- [O problema da separação](#)
- [O algoritmo da separação](#)
- [Quicksort básico](#)
- [Animações do Quicksort](#)
- [Desempenho do Quicksort básico](#)
- [Altura da pilha de execução do Quicksort](#)

O problema da separação

O núcleo do algoritmo Quicksort é o seguinte problema da separação (= *partition subproblem*), que formularemos de maneira propositalmente vaga:

rearranjar um vetor $v[p..r]$ de modo que todos os elementos pequenos fiquem na parte esquerda do vetor e todos os elementos grandes fiquem na parte direita.

O ponto de partida para a solução desse problema é a escolha de um *pivô* (ou *fulcro*), digamos c . Os elementos do vetor que forem maiores que c serão considerados grandes e os demais serão considerados pequenos. É importante escolher c de tal modo que cada parte do vetor rearranjado seja estritamente menor que o vetor todo. O desafio é resolver o problema da separação rapidamente e sem usar um vetor auxiliar como “espaço de trabalho”.

O problema da separação admite muitas formulações concretas. Eis uma primeira: rearranjar $v[p..r]$ de modo que tenhamos $v[p..j] \leq v[j+1..r]$ para algum j em $p..r-1$. (Nessa formulação, o pivô não é explícito.) Eis uma segunda formulação: rearranjar $v[p..r]$ de modo que

$$v[p..j-1] \leq v[j] < v[j+1..r]$$

para algum j em $p..r$. (Aqui, $v[j]$ é o pivô.) Exemplo:

j									
666	222	111	777	555	444	555	777	999	888

Neste capítulo, usaremos a segunda formulação do problema da separação.

Exercícios 1

1. POSITIVOS E NEGATIVOS. Escreva uma função que rearranje um vetor $v[p..r]$ de números inteiros de modo que tenhamos $v[p..j-1] \leq 0$ e $v[j..r] > 0$ para algum j em $p..r+1$. Faz sentido exigir que j esteja em $p..r$? Procure escrever uma função *rápida* que não use vetor auxiliar. Repita o exercício depois de trocar “ $v[j..r] > 0$ ” por “ $v[j..r] \geq 0$ ”. Faz sentido exigir que $v[j]$ seja 0?
2. LISTA. Suponha dada uma [lista encadeada](#) que armazena números inteiros estritamente positivos. Escreva uma função que transforme a lista em duas: a primeira contendo os números pares e a segunda contendo os ímpares.
3. Critique a seguinte tentativa de resolver o problema da separação:

```
int sep (int v[], int p, int r) {
    int j = r;
    for (int i = r-1; i >= p; i--)
        if (v[i] > v[r]) {
            int t = v[r]; v[r] = v[i]; v[i] = t;
            j = i; }
    return j; }
```

4. Critique a seguinte solução do problema da separação:

```
int sep (int v[], int p, int r) {
    int w[1000], i = p, j = r, c = v[r];
    for (int k = p; k < r; ++k)
        if (v[k] <= c) w[i++] = v[k];
        else w[j--] = v[k];
    // agora i == j
    w[j] = c;
    for (int k = p; k <= r; ++k) v[k] = w[k];
    return j; }
```

5. Um programador inexperiente afirma que a seguinte função resolve a primeira formulação do problema da separação, ou seja, rearranja qualquer vetor $v[p..r]$ (com $p < r$) de tal forma que $v[p..i] \leq v[i+1..r]$ para algum i em $p..r-1$.

```

int sep (int v[], int p, int r) {
    int i = p, j = r;
    int q = (p + r)/2;
    do {
        while (v[i] < v[q]) ++i;
        while (v[j] > v[q]) --j;
        if (i <= j) {
            int t = v[i], v[i] = v[j], v[j] = t;
            ++i, --j; }
    } while (i <= j);
    return i; }

```

Mostre um exemplo em que essa função não dá o resultado prometido. E se trocarmos “return i” por “return i-1”? É possível fazer algumas poucas correções de modo que a função dê o resultado esperado?

6. ★ Digamos que um vetor $v[p..r]$ está *separado* se existe j em $p..r$ tal que $v[p..j-1] \leq v[j] < v[j+1..r]$. Escreva um algoritmo que decida se $v[p..r]$ está separado. Em caso afirmativo, o seu algoritmo deve devolver o índice j .

O algoritmo da separação

Eis como o [livro de Cormen, Leiserson, Rivest e Stein](#) implementa o algoritmo da separação. (A implementação é atribuída a N. Lomuto.) A função resolve a [segunda formulação](#) do problema da separação:

```

// Recebe vetor v[p..r] com p <= r. Rearranja
// os elementos do vetor e devolve j em p..r
// tal que v[p..j-1] <= v[j] < v[j+1..r].

```

```

static int
separa (int v[], int p, int r) {
    int c = v[r]; // pivô
    int t, j = p;
    for (int k = p; /*A*/ k < r; ++k)
        if (v[k] <= c) {
            t = v[j], v[j] = v[k], v[k] = t;
            ++j;
        }
    t = v[j], v[j] = v[r], v[r] = t;
    return j;
}

```

(A palavra-chave `static` está aí apenas para indicar que `separa` é uma função auxiliar e não deve ser invocada diretamente pelo usuário final do Quicksort.)

Para mostrar que a função `separa` está correta, basta verificar que no início de cada iteração — ou seja, a cada passagem pelo ponto A — temos a seguinte configuração:

p			j		k			r	
$\leq c$	$\leq c$	$\leq c$	$> c$	$> c$	?	?	?	?	c

(Note que podemos ter $j == p$ bem como $k == j$, ou seja, a metade esquerda ou a metade direita do vetor podem estar vazias.) Mais exatamente, valem os seguintes [invariantes](#): no início de cada iteração,

1. $p \leq j \leq k$,
2. $v[p..j-1] \leq c$,
3. $v[j..k-1] > c$ e
4. $v[p..r]$ é uma permutação do vetor original.

Na última passagem pelo ponto A, teremos $k == r$ e portanto a seguinte configuração:

p					j				k	
≤ c	≤ c	≤ c	≤ c	≤ c	> c	> c	> c	> c	c	

Assim, na fim da execução da função, teremos $p \leq j \leq r$ e $v[p..j-1] \leq v[j] < v[j+1..r]$, como prometido.

p					j				r	
≤ c	≤ c	≤ c	≤ c	≤ c	c	> c	> c	> c	> c	

[Gostaríamos](#) j que ficasse a meio caminho entre p e r, mas devemos estar preparados para aceitar os casos extremos em que j vale p ou r.

Desempenho. Quanto tempo a função separa consome? O consumo de tempo é proporcional ao número de comparações entre elementos do vetor. Não é difícil perceber que esse número é proporcional ao tamanho $r - p + 1$ do vetor. A função é, portanto, [linear](#).

Exercícios 2

- Seja $v[0..15]$ o vetor 33 22 55 33 44 22 99 66 55 11 88 77 33 88 66 66. Execute `partition(v, 0, 15)`.
- CASO EXTREMO. Qual o resultado de `separa(v, p, r)` quando $p == r$?
- CASOS EXTREMOS. Qual o resultado de `separa(v, p, r)` quando os elementos de $v[p..r]$ são todos iguais? E quando $v[p..r]$ é crescente? E quando $v[p..r]$ é decrescente? E quando cada elemento de $v[p..r]$ tem um de dois possíveis valores?
- INVARIANTES. Prove que os invariantes de `separa` enunciados [acima](#) estão de fato corretos.
- VERSÃO RECURSIVA. Escreva uma versão recursiva da função `separa`.
- PRIMEIRO ELEMENTO COMO PIVÔ. Reescreva o código de `separa` de modo que $v[p]$ seja usado como de pivô. (Uma possibilidade é intercambiar $v[p]$ e $v[r]$ e depois executar `separa`. Mas é mais instrutivo reescrever o código.)
- A função `separa` produz um rearranjo [estável](#) do vetor, ou seja, preserva a ordem relativa de elementos de mesmo valor?
- IMPLEMENTAÇÃO GRIES DA SEPARAÇÃO (veja *The Science of Programming* de D. Gries). Analise e discuta a seguinte implementação do algoritmo de separação. Mostre que essa implementação é equivalente à função `separa` [de Lomuto](#). [Solução: ./solucoes/quick8.html]

```
int separa_Gries (int v[], int p, int r) {
    int c = v[p], i = p+1, j = r;
    while (true) {
        while (i <= r && v[i] <= c) ++i;
        while (c < v[j]) --j;
        if (i >= j) break;
        int t = v[i], v[i] = v[j], v[j] = t;
        ++i; --j; }
    v[p] = v[j], v[j] = c;
    return j; }
```

- Critique a seguinte variante da [função separa_Gries](#):

```
int sep (int v[], int p, int r) {
    int c = v[p], i = p+1, j = r;
    while (i <= j) {
        if (v[i] <= c) ++i;
        else {
            int t = v[i], v[i] = v[j], v[j] = t;
            --j; } }
    v[p] = v[j], v[j] = c;
    return j; }
```

10. [ROBERTS](#). Critique a seguinte solução do problema da separação:

```
int separa_R (int v[], int p, int r) {
    int c = v[p], i = p+1, j = r;
    while (1) {
        while (i < j && c < v[j]) --j;
        while (i < j && v[i] <= c) ++i;
        if (i == j) break;
        int t = v[i], v[i] = v[j], v[j] = t; }
    if (v[j] > c) return p;
    v[p] = v[j], v[j] = c;
    return j; }
```

11. DESAFIO. Escreva uma solução do problema da separação que devolva um índice j tal que $(r-p)/10 \leq j-p \leq 9(r-p)/10$.

12. Um vetor é *binário* se cada um de seus elementos vale 0 ou 1. Escreva uma função que rearranje um vetor binário $v[0..n-1]$ em ordem crescente e consuma tempo proporcional ao tamanho do vetor.

Quicksort básico

Agora que resolvemos o problema da separação, podemos cuidar do Quicksort propriamente dito. O algoritmo usa a estratégia da [divisão e conquista](#) e tem a aparência de um [Mergesort](#) “ao contrário”:

```
// Esta função rearranja qualquer vetor
// v[p..r] em ordem crescente.
```

```
void
quicksort (int v[], int p, int r)
{
    if (p < r) {
        int j = separa (v, p, r); // 1
        quicksort (v, p, j-1);    // 2
        quicksort (v, j+1, r);    // 3
    }
}
```

Se $p \geq r$ (essa é a base recursão) não é preciso fazer nada. Se $p < r$, o código reduz a [instância](#) $v[p..r]$ do problema ao par de instâncias $v[p..j-1]$ e $v[j+1..r]$. Como $p \leq j \leq r$, essas duas instâncias são estritamente menores que a instância original. Assim, por hipótese de indução, $v[p..j-1]$ estará em ordem crescente no fim da linha 3 e $v[j+1..r]$ estará em ordem crescente no fim da linha 4. Como $v[p..j-1] \leq v[j] < v[j+1..r]$ graças à função *separa*, o vetor todo estará em ordem crescente no fim da linha 4, como promete a documentação da função.

Podemos eliminar a segunda chamada recursiva (linha 4) e trocar o *if* por um *while*. Isso produz uma função equivalente ao *quicksort* acima:

```
void qcksrt (int v[], int p, int r) {
    while (p < r) {
        int j = separa (v, p, r);
        qcksrt (v, p, j-1);
        p = j + 1;
    }
}
```

Exercícios 3

1. Que acontece se trocarmos " $p < r$ " por " $p \leq r$ " na linha 1 do quicksort? Que acontece se trocarmos " $p < r$ " por " $p \neq r$ " na linha 1?
2. Que acontece se trocarmos " $j-1$ " por " j " na linha 3 do código? Que acontece se trocarmos " $j+1$ " por " j " na linha 4?
3. PEGADINHA. Quais são os [invariantes](#) da função quicksort?
4. Seja $v[1..4]$ o vetor 77 55 33 99. Se executar quicksort (v, p, r), você verá a seguinte sequência de invocações:

```
quicksort (v,1,4)
  quicksort (v,1,2)
    quicksort (v,1,0)
    quicksort (v,2,2)
  quicksort (v,4,4)
```

Repita o exercício com o vetor $v[1..6] = 55\ 44\ 22\ 11\ 66\ 33$.

5. Escreva uma função com protótipo `quick_sort (int v[], int n)` que reorganize um vetor $v[0..n-1]$ (atenção aos índices!) em ordem crescente. (Basta invocar quicksort da maneira correta.)
6. A função quicksort produz uma ordenação [estável](#) do vetor?
7. Discuta a seguinte implementação do algoritmo Quicksort (supondo $p < r$):

```
void qsrt (int v[], int p, int r) {
    int j = separa (v, p, r);
    if (p < j-1) qsrt (v, p, j-1);
    if (j+1 < r) qsrt (v, j+1, r);
}
```

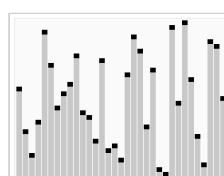
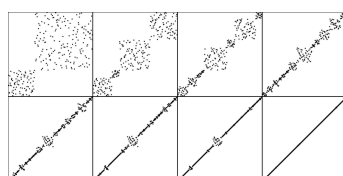
8. ★ FORMULAÇÃO ALTERNATIVA DA SEPARAÇÃO. Suponha dada uma função que resolve a primeira das formulação do problema da separação dadas [acima](#) (ou seja, reorganiza $v[p..r]$ e devolve j tal que $v[p..j] \leq v[j+1..r]$.) Escreva uma versão do algoritmo Quicksort que use essa função de separação. Que restrições devem ser impostas sobre j ?
9. ★ VERSÃO ITERATIVA. Escreva uma versão não recursiva do algoritmo Quicksort. [Solução: ./solucoes/quick-it.html]
10. ★ QUICKSORT ALEATORIZADO. Escreva uma [versão aleatorizada](#) (= *randomized*) do algoritmo Quicksort.
11. TESTE DE CORREÇÃO. Escreva um programa que teste, experimentalmente, a correção de sua implementação do algoritmo Quicksort. (Veja the [exercício análogo para Insertionsort](#).)

Animações do Quicksort

A animação à direita (copiada da [Simon Waldherr / Golang Sorting Visualization](#)) mostra a aplicação do Quicksort a um vetor $v[0..79]$ de números positivos. Cada elemento $v[i]$ do vetor é representado pelo ponto de coordenadas $(i, v[i])$.



A primeira das figuras abaixo (copiada da [Wikimedia](#)) mostra a aplicação do Quicksort a uma permutação $v[0..249]$ do conjunto $0..249$. A figura mostra o estado do vetor em 8 momentos da ordenação. A segunda figura (copiada da [Wikipedia](#)) mostra a aplicação do Quicksort a uma permutação $v[0..32]$ do conjunto $0..32$.



Há muitas outras animações do Quicksort. Veja alguns exemplos:

- [Animation of Quicksort](#), de C. Parsons, on YouTube.
- [Animation of Quicksort](#) produzido por Mike Bostock. [Link sugerido por Yoshiharu Kohayakawa.] O vetor é representado por uma “vassoura”. Cada cerda da “vassoura” é um elemento do vetor e a inclinação da cerda é o valor do elemento. Seguem algumas apresentações autônomas da animação:
 - [Quicksort](#). O pivô está destacado em vermelho. (Veja [o código](#).)
 - [Quicksort IV](#). Mostra o bloco ativo da partição em preto e o pivô ativo em vermelho. O algoritmo intercambia $v[(p+r)/2]$ e $v[r]$ antes de fazer a separação, ou seja, usa o valor do elemento do meio do vetor como pivô. (Veja [o código](#).)
 - [Quicksort II](#). Uma visualização estática da [versão iterativa](#). Cada linha corresponde ao estado do vetor antes de uma separação. As linhas vermelhas representam os pivôs. Recarregue para variar os dados. (Veja [o código](#).)
 - [Quicksort V](#). Uma visualização estática usando fitas coloridas que se entrelaçam. Cada linha representa o estado do vetor depois de uma operação de separação. Recarregue para mudar os dados. (Veja [o código](#).)
 - [Quicksort VI](#). Uma versão animada de Quicksort V. (Veja [o código](#).)
- [Quick-sort com dança folclórica húngara](#), Universidade Sapiientia (Romênia).
- Animação de [15 algoritmos de ordenação](#), de Timo Bingmann, no YouTube.
- [Comparison Sorting Algorithms](#), de David Galles (University of San Francisco).
- [Sorting Algorithms Animations](#) da Toptal.

Desempenho do Quicksort básico

Não é difícil perceber que o consumo de tempo da função `quicksort` é proporcional ao número de comparações entre elementos do vetor. Se o índice j devolvido por `separa` estiver [sempre mais ou menos a meio caminho](#) entre p e r , o número de comparações será aproximadamente $n \log n$, sendo n o tamanho $r-p+1$ do vetor. Por outro lado, se o vetor já estiver ordenado ou quase ordenado, o número de comparações será aproximadamente

$$n^2.$$

Portanto, o pior caso do `quicksort` não é melhor que o dos [algoritmos elementares](#). Felizmente, o pior caso é muito raro: *em média*, o consumo de tempo do `quicksort` é proporcional a

$$n \log n.$$

(Veja detalhes na página [Análise do Quicksort](#) do meu sítio *Análise de Algoritmos*.) Portanto, o algoritmo é [linearítmico](#) em média e [quadrático](#) no pior caso.

Exercícios 4

1. VERSÃO “TRUNCADA”. Escreva uma versão do algoritmo Quicksort com *cutoff* para vetores pequenos: quando o vetor a ser ordenado tiver menos que M elementos, a ordenação passa a ser feita pelo [algoritmo Insertionsort](#). O valor de M pode ficar entre 10 e 20. (Esse truque é usado na prática porque o algoritmo Insertionsort é mais rápido que o Quicksort puro quando o vetor é pequeno. O fenômeno é muito comum: algoritmos sofisticados são tipicamente mais lentos que algoritmos simplórios quando o volume de dados é pequeno.)
2. A seguinte função [contribuição de Pedro Rey] promete rearranjar o vetor $v[p..r]$ em ordem crescente. Mostre que a função está correta. Estime o consumo de tempo da função.

```
void psort (int v[], int p, int r) {
    if (p >= r) return;
```

```

if (v[p] > v[r]) {
    int t = v[p], v[p] = v[r], v[r] = t; }
psort (v, p, r-1);
psort (v, p+1, r); }

```

3. ★ OUTRAS VERSÕES DO QUICKSORT. Procure outras versões do algoritmo Quicksort nos livros e na rede WWW. Discuta as diferenças em relação ao código acima. Qual a forma do [problema da separação](#) resolvido por cada versão? [Solução: ./solucoes/quick7.html]
4. TESTE DE DESEMPENHO. Escreva um programa que cronometre sua implementação do Quicksort. (Veja [exercício análogo para o Mergesort](#).)

Altura da pilha de execução do Quicksort

Na [versão básica do Quicksort](#), o código cuida imediatamente do segmento $v[p..j-1]$ e trata do segmento $v[j+1..r]$ somente depois que $v[p..j-1]$ estiver ordenado. Dependendo do valor de j nas sucessivas chamadas da função, a [pilha de execução](#) pode crescer muito, atingindo altura igual ao tamanho do vetor. (Isso acontece, por exemplo, se o vetor estiver em ordem decrescente.) Esse fenômeno pode esgotar o espaço de memória. Para controlar o crescimento da pilha de execução é preciso tomar duas providências:

1. cuidar primeiro do $\triangle$ menor dos segmentos $v[p..j-1]$ e $v[j+1..r]$ e
2. eliminar a segunda chamada recursiva da função, [trocando o if por um while](#).

Se adotarmos essas providências, o tamanho do segmento que está no topo da pilha de execução será menor que a metade do tamanho do segmento que está logo abaixo na pilha. De modo mais geral, o segmento que está em qualquer das posições da pilha de execução será menor que metade do segmento que está imediatamente abaixo. Assim, se a função for aplicada a um vetor com n elementos, a altura da pilha não passará de $\log n$.

```

// A função rearranja o vetor v[p..r]
// em ordem crescente.

```

```

void
quickSort (int v[], int p, int r)
{
    while (p < r) {
        int j = separa (v, p, r);
        if (j - p < r - j) {
            quickSort (v, p, j-1);
            p = j + 1;
        } else {
            quickSort (v, j+1, r);
            r = j - 1;
        }
    }
}

```

Exercícios 5

1. Suponha que a seguinte função é aplicada a um vetor com n elementos. Mostre que a altura da pilha de execução pode passar de $\log n$.

```

void quicks (int v[], int p, int r) {
    if (p < r) {
        int j = separa (v, p, r);
        if (j - p < r - j) {
            quicks (v, p, j-1);

```



```
        quicks (v, j+1, r); }  
    else {  
        quicks (v, j+1, r);  
        quicks (v, p, j-1); } } }
```

2. Familiarize-se com a função [qsort](#) da biblioteca `stdlib`.

Veja o verbete [Quicksort](#) na Wikipedia.

Veja minha página [Análise do Quicksort](#).

Veja capítulo 11 do [Programming Pearls](#) de Jon Bentley.

Veja a página [Quicksort](#) no Cprogramming.com.

Atualizado em 2019-03-01

<https://www.ime.usp.br/~pf/algoritmos/>

© *Paulo Feofiloff*

[DCC-IME-USP](#)